


```

9      "a": read-only, get value from eax/ax/al
10     "d": set value to edx/dx/dl
11 */
12
13 unsigned char inb(unsigned short int port_from){
14     unsigned char _in_value;
15     __asm__ __volatile__ ("inb %w1,%0":"=a"(_in_value):"Nd"(port_from));
16     return _in_value;
17 }
18
19 void outb (unsigned short int port_to, unsigned char value){
20     __asm__ __volatile__ ("outb %b0,%w1":"=a" (value),"Nd" (port_to));
21 }

```

2. vga.c

这里我使用 `void update_pointer(void)` 函数更新vga指针

`void update_cursor(void)` 函数通过当前行值`cur_cline`与列值`cur_column`回写光标

`void scroll_screen(void)` 函数实现滚屏

```

1  /*
2   * 本文件实现vga的相关功能，清屏和屏幕输出
3   * clear_screen和append2screen必须按照如下借口实现
4   * 可以增加其他函数供clear_screen和append2screen调用
5   */
6  extern void outb (unsigned short int port_to, unsigned char value);
7  extern unsigned char inb(unsigned short int port_from);
8
9  //VGA字符界面规格：25行80列
10 #define VGA_WIDTH 80
11 #define VGA_HEIGHT 25
12
13 //VGA显存初始地址为0xB8000
14
15 short cur_line = 0;
16 short cur_column = 0; //当前光标位置
17 char * vga_init_p = (char *)0xB8000;
18 char * vga_current_p = (char *)0xB8000;
19
20 /*
21
22 *不必局限于以下框架，可以自定义接口来实现，只需要保证append2screen接口能够
23 被正确调用即可
24
25 */
26 void update_pointer(void){ //更新vga指针
27     vga_current_p = vga_init_p + (cur_line * VGA_WIDTH + cur_column) * 2; //
    一个字符占 16bit，故乘2
28 }
29
30 void update_cursor(void){ //通过当前行值cur_cline与列值cur_column回写光标
31
32     unsigned short position = cur_line * VGA_WIDTH + cur_column;
33
34     outb(0x3D4, (unsigned char)0xE); // 写入高 8 位
35     outb(0x3D5, (unsigned char)((position >> 8) & 0xFF));

```

```

36
37     outb(0x3D4, (unsigned char)0xF); // 写入低 8 位
38     outb(0x3D5, (unsigned char)(position & 0xFF));
39 }
40
41 void scroll_screen(void){//屏幕向上滚动一行
42
43     for(int i=0 ; i< (VGA_HEIGHT-1) * VGA_WIDTH ; i++)
44     {
45         *(vga_init_p + i * 2) = *(vga_init_p + (i + VGA_WIDTH) * 2);
46         *(vga_init_p + i * 2 + 1) = *(vga_init_p + (i + VGA_WIDTH) * 2 + 1);
47     }
48     for(int i=0 ; i< VGA_WIDTH ; i++)
49     {
50         *(vga_init_p + (VGA_HEIGHT - 1) * VGA_WIDTH * 2 + i * 2) = ' ';
51         *(vga_init_p + (VGA_HEIGHT - 1) * VGA_WIDTH * 2 + i * 2 + 1) = 0x07;
52     }
53     update_cursor();
54 }
55
56 void clear_screen(void) { //清屏
57     for(int i=0 ; i< VGA_HEIGHT * VGA_WIDTH ; i++)
58     {
59         *(vga_init_p + i * 2) = ' ';
60         *(vga_init_p + i * 2 + 1) = 0x07;
61     }
62     cur_line = 0;
63     cur_column = 0;
64     update_cursor();
65     update_pointer();
66 }
67
68 void append2screen(char *str,int color){ //将str输出到vga屏幕
69     int i = 0;
70     while(str[i] != '\0')
71     {
72         if(str[i] == '\n') //换行
73         {
74             cur_line++;
75             cur_column = 0;
76         }
77         else
78         {
79             *(vga_current_p) = str[i];
80             *(vga_current_p + 1) = color;
81             cur_column++;
82         }
83         if(cur_column >= VGA_WIDTH) //换行
84         {
85             cur_line++;
86             cur_column = 0;
87         }
88         if(cur_line >= VGA_HEIGHT) //滚屏
89         {
90             scroll_screen();
91             cur_line--;
92             cur_column = 0;
93         }

```

```

94         i++;
95         update_cursor();
96         update_pointer();
97     }
98 }

```

3.uart.c

```

1  /*
2   * 与 UART 相关的输出
3   * 调用inb和outb函数，实现下面的uart的三个函数
4   */
5  extern unsigned char inb(unsigned short int port_from);
6  extern void outb (unsigned short int port_to, unsigned char value);
7
8  #define uart_base 0x3F8
9
10 void uart_put_char(unsigned char c){
11     outb(uart_base,c);
12 }
13
14 unsigned char uart_get_char(void){
15     return inb(uart_base);
16 }
17
18 void uart_put_chars(char *str){
19     while(*str != '\0')
20     {
21         uart_put_char(*str);
22         str++;
23     }
24 }

```

第三部分 编译过程进行说明

src文件夹结构如下：

```

1  .
2  ├── Makefile
3  ├── Makefile:Zone.Identifier
4  ├── multibootheader
5  │   ├── multibootHeader.S
6  │   └── multibootHeader.S:Zone.Identifier
7  ├── myOS
8  │   ├── Makefile
9  │   ├── Makefile:Zone.Identifier
10  │   ├── dev
11  │   │   ├── Makefile
12  │   │   ├── Makefile:Zone.Identifier
13  │   │   ├── uart.c
14  │   │   ├── uart.c:Zone.Identifier
15  │   │   ├── vga.c
16  │   │   └── vga.c:Zone.Identifier
17  │   └── i386
18  │       ├── Makefile
19  │       └── Makefile:Zone.Identifier

```

```

20 | | | └─ io.c
21 | | | └─ io.c:Zone.Identifier
22 | | | └─ io.h
23 | | | └─ io.h:Zone.Identifier
24 | | └─ myOS.ld
25 | | └─ myOS.ld:Zone.Identifier
26 | | └─ osStart.c
27 | | └─ osStart.c:Zone.Identifier
28 | | └─ printk
29 | | | └─ Makefile
30 | | | └─ Makefile:Zone.Identifier
31 | | | └─ myPrintk.c
32 | | | └─ myPrintk.c:Zone.Identifier
33 | | | └─ vsprintf.c
34 | | | └─ vsprintf.c:Zone.Identifier
35 | | └─ start32.S
36 | | └─ start32.S:Zone.Identifier
37 | └─ output
38 | | └─ multibootheader
39 | | | └─ multibootHeader.o
40 | | └─ myOS
41 | | | └─ dev
42 | | | | └─ uart.o
43 | | | | └─ vga.o
44 | | | └─ i386
45 | | | | └─ io.o
46 | | | └─ osStart.o
47 | | | └─ printk
48 | | | | └─ myPrintk.o
49 | | | | └─ vsprintf.o
50 | | | └─ start32.o
51 | | └─ myOS.elf
52 | | └─ userApp
53 | | | └─ main.o
54 | └─ source2run.sh
55 | └─ source2run.sh:Zone.Identifier
56 | └─ userApp
57 | | └─ Makefile
58 | | └─ Makefile:Zone.Identifier
59 | | └─ main.c
60 | | └─ main.c:Zone.Identifier

```

运行source2run.sh 脚本启动。

第四部分 运行结果

测试main函数如下：

```

1  /*
2   * 请确保以下的代码可以原封不动的成功编译和执行
3   * 助教测试时会替换相应的参数
4   * 不需要修改
5   */
6  extern int myPrintf(int color,const char *format, ...);
7  extern int myPrintk(int color,const char *format, ...);
8
9  void myMain(void){

```

```

10     int i;
11     /*NOTE:用户使用myPrintf而不使用myPrink接口*/
12     myPrintf(0x7,"main\n");
13     for (i=1;i<30;i++) myPrintf(i,"%d\n",i);
14
15     myPrintf(0x7,"asildhasdjlkasjdl;kasjdl;asjd;ljasld;jasl;djasl;djakfhiowhvfjsj
16     f;klasjcl;sdjpofajsdofjsd;lfk123456789\n");
17     myPrintf(0x7,"liyiPB22051031\n");
18     return;
19 }
20

```

运行结果如下:

```

1  rm -rf output
2  ld -n -T myOS/myOS.ld output/multibootheader/multibootHeader.o
   output/myOS/start32.o output/myOS/osStart.o output/myOS/dev/uart.o
   output/myOS/dev/vga.o output/myOS/i386/io.o output/myOS/printk/myPrintk.o
   output/myOS/printk/vsprintf.o output/userApp/main.o -o output/myOS.elf
3  ld: warning: output/myOS/start32.o: missing .note.GNU-stack section implies
   executable stack
4  ld: NOTE: This behaviour is deprecated and will be removed in a future
   version of the linker
5  ld: warning: output/myOS.elf has a LOAD segment with RWX permissions
6  make succeed

```

成功显示, 且光标位置正确。